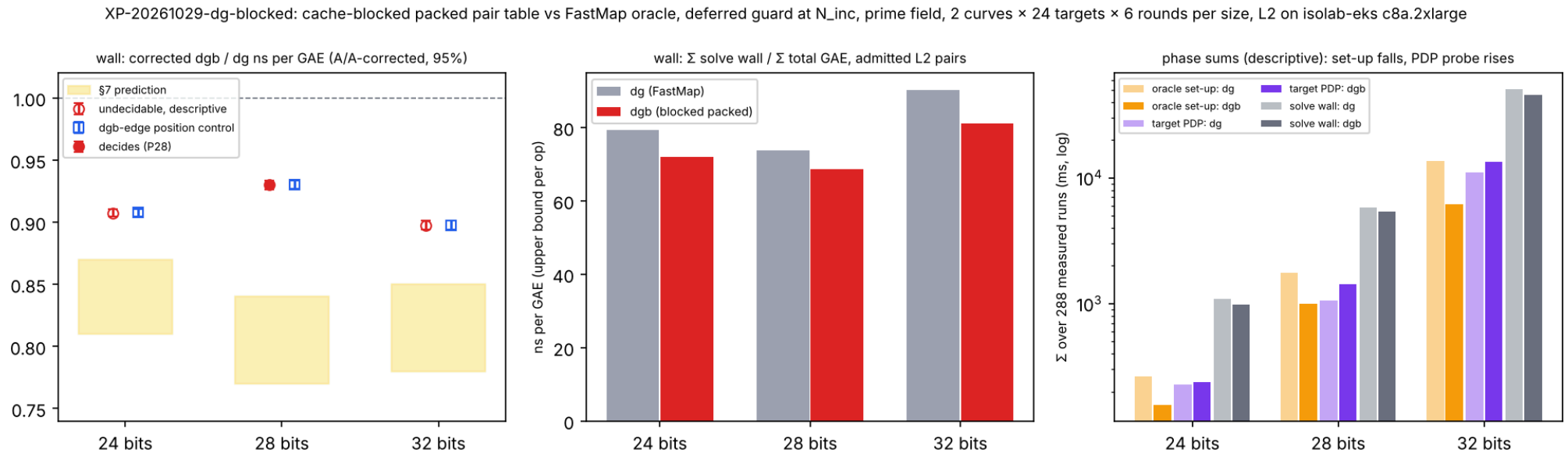
Prime-field IC: a cache-blocked packed pair table lowers wall time per charged operation at 28 bits (XP-20261029-dg-blocked)

Frozen protocol: `backlog/XP-20261029-dg-blocked/PROTOCOL.md` (commit cd04cfd2d1, approved with edits E1–E10 in `APPROVAL.md`), plus the Coordinator ruling `cards/RULING-P2.md`, which adds condition A1. Verdict commit: bf899979a6 (`VERDICT.md`).

Question. The incumbent prime-field oracle `mitm:negation_folded=1` ("dg") keeps its pair table in a scalar FastMap. `mitm-prime-blocked:m=2` ("dgb") builds the same table as a PackedPairTable, cache-blocked. Both arms run `targets=stride-renew-deferred-guard` at the incumbent factor base N_inc. They charge identical work, so the question is whether the packed layout costs less real time. This study measures it as wall time per charged GAE, at L2, with a native A/A correction.

Every ratio and interval in the decision and counted tables is read from `ecbench compare` or `ecbench band` output, and every phase sum from the card's `phase_sums.json`, in `P<b>/{wall,counted}/analysis/`. The percentages and fractions in the phase discussion are descriptive arithmetic on those sums; the validation figures come from the validator report.



Decision (§6.2)

Statistic: the `wall` block of `ecbench compare --a dg --b dgb --weight 0` on the wall session. `corrected_ratio` = `ratio_b_over_a / aa_ratio`, with A/A `dg-aa / dg`. Each size has 288 pairs, from 2 curves × 24 targets × 6 rounds.

b	N_inc	dg ns/GAE	dgb ns/GAE	A/A ratio	**corrected dgb / dg**	§7 prediction	reading
24	162	79.34	72.03	1.0004	0.9075 [0.9045, 0.9105]	0.81–0.87	**undecidable** (descriptive only)
28	407	73.83	68.66	1.0001 [0.9991, 1.0012]	**0.9300 [0.9266, 0.9333]**	0.77–0.84	**improvement** (smaller than predicted)
32	1024	90.32	81.07	1.0001	0.8975 [0.8939, 0.9011]	0.78–0.85	**undecidable** (descriptive only)

28 bits. All of §6.1 conditions 1–9 and A1 hold:

- the wall job exits 0;
- all 1,344 executions are L2;
- the native A/A selection is `includes arm A`, with 288 pairs;

- the condition-9 identity is exact (relative difference 0.0);
- condition 8 holds: 336/336 records are identical in charged work for dg against dgb and for dg against dgb-edge, warm-ups included, and the counted session equals the wall session;
- A1 holds: every prime is below 2^32 − 1 and every signed-point count is at most 65,536, so the blocked oracle ran in the packed regime and did not fall back to small maps.

24 and 32 bits are undecidable. The frozen rule makes a size undecidable when any wall execution falls below L2. Such a session exits 42 and is reported, not rerun (APPROVAL.md "Residual risks").

- **24 bits.** Two executions were graded L1 for a solve under the 1 ms minimum: one `dgb` (913.0 µs) and one `dgb-edge` (903.9 µs). One counted `dgb` execution (911.2 µs) was also L1, so the counted exit-42 clause fails at 24 bits as well.
- **32 bits.** One execution was graded L1: `dg-aa` seq 1313, a 195.1 ms solve with "3 foreign task slices on the run CPU during the solve (0 µs)".
- Conditions 1, 5, 6, 7 and 9 fail at both sizes; conditions 2, 3, 4, 8 and A1 hold.
- Their ratios are plotted hollow and decide nothing.

The ns/GAE figures are upper bounds per operation. Both arms carry unpriced native work.

Not relabelling, not accounting. dg and dgb charge identical work record by record at every size, so their counted S is identical (see "Counted units" below). The wall difference is executed native work: a different memory layout for the same table.

Arm positions (validator, `validation/work/recompute.json`, `sizes.P<b>.positions`; identical at 24, 28 and 32 bits over the 288 measured blocks):

arm	block positions	blocks per position
`dg`	0 and 3	144 each
`dg-aa`	1 and 2	144 each
`dgb`	1 and 2	144 each
`dgb-edge`	0 and 3	144 each

There are 144 forward and 144 reversed blocks; `dgb` follows `dg-aa` in 144 blocks and precedes it in 144.

Position control. `dgb-edge` runs the same oracle as `dgb` at the edge of the alternate block. Its corrected ratio is 0.9302 [0.9268, 0.9336] at 28 bits, and 0.9077 and 0.8977 at 24 and 32 bits (descriptive), so the block position does not explain the gain.

Where the time went (descriptive; decides nothing)

Phase sums over the 288 measured runs, from `P<b>/wall/analysis/phase_sums.json`:

b	oracle set-up dg → dgb (ms)	target PDP dg → dgb (ms)	relations phase dg → dgb (ms)	solve wall dg → dgb (ms)
24	262 → 156	228 → 239	744 → 752	1084 → 984
28	1769 → 1000	1062 → 1428	3800 → 4161	5766 → 5363
32	13627 → 6150	10998 → 13353	36367 → 38616	50848 → 45639

The blocked build roughly halves the oracle set-up, but probing the packed table in the target PDP phase is slower than probing FastMap (+5%, +34% and +21%). The probe cost gives back part of the set-up gain. Together with a set-up saving smaller than predicted at 24 and 28 bits (0.594 and 0.565 of dg's, against 0.25–0.5), it explains why every ratio missed its prediction: with the target-PDP phase unchanged, the ratios would still be 0.898, 0.867 and 0.851 (validator, descriptive, taking phase times as additive). Only at 32 bits, where the set-up fell to 0.451, is the PDP rise nearly the whole miss. E6 flagged the probe risk. PR #2844's table-floor micro-benchmark (outside this evidence, not checked by the validator) points the same way: a flat one-region table is 2.3–2.9× slower on misses.

Next lever (untested): keep the blocked build and probe through the map layout, or prefetch packed slots in the PDP phase.

Counted units and distance to matched rho (counted session)

`Σ S_arm / Σ S_rho-neg` with a 95% cluster bootstrap, where `rho-neg` is `rho.negation` on the same targets in the same session. This is the distance remaining, not a gate. dg and dgb are identical at every weight.

b	arm	w = 0	w = 0.1	w = 1
24	dg = dgb	12.96 [11.97, 14.08]	12.45 [11.48, 13.53]	10.03 [9.22, 10.96]
28	dg = dgb	21.79 [20.12, 23.69]	20.66 [19.07, 22.46]	15.48 [14.25, 16.88]
32	dg = dgb	33.07 [30.36, 36.17]	31.49 [28.90, 34.44]	24.35 [22.34, 26.66]

The 24-bit row is descriptive because that size's counted exit-42 clause fails. At 28 and 32 bits the counted jobs exited 42 under the allowed clause: every below-L2 entry is a waived `rho-neg` or `rho-neg-aa` solve under 1 ms, 273 at 28 bits and 10 at 32 bits.

Validation

Independent validator **VAL-20261029-33a286**: **28 bits passes with caveats; 24 and 32 bits fail** (it agrees they are undecidable). The files are `validation/validator-report.json` (sha256 9e5e1d34…), `validation/replay-certificate.json` (d75487e8…) and `VALIDATION.json`, which names both and the matched `rho.negation` reference with its §5a band for the two 28-bit rows. `validation/SHA256SUMS` covers the validator's code, frozen inputs, work files and replay logs.

What it checked, with its own code:

- 337 headline numbers recomputed, none off by more than 1e-15; 8,064 record seals recomputed, 0 failures;
- every evidence file byte-equal to its S3 pod original, one pod per job, no reruns; the launch receipts equal the S3 launcher receipts;
- 6,912/6,912 runs replayed identically by the same binary on another host (Intel);
- `dgb` and `dgb-edge` match `dg` in every charged field (recovered log, GAE, unpriced and phase counts, seeds, bitops, factor-base hashes) on all 336 records per size; only wall time differs, plus peak memory at 32 bits (80.3 → 61.9 MB);
- A1 from the `ic_boundary.rs` fallback condition: every `dgb` record ran in the packed layout;
- `dgb` and `dg-aa` balanced across block positions (144/144);
- Aut(E) = {±1} on every curve, so `rho.negation` is the strongest rho.

Its caveats:

- **Curves.** The 28-bit per-curve corrected ratios differ beyond their intervals: `t9805` 0.9379 [0.9333, 0.9425], `tm3463` 0.9212 [0.9168, 0.9257]. Both are below 1. Target-clustered intervals stay below 1 ([0.92610, 0.93382] linearised, [0.92703, 0.93276] bootstrap).
- **Host.** One host class (AMD EPYC 9R45); wall time cannot be replayed.
- **Scope.** Packed layout only; the small-maps branch of the same oracle string is untested.
- **Prediction.** Missed: set-up fell to 0.565 of the scalar build (predicted 0.25–0.5), and the PDP probe rose by 34%.

What ran

b	wall job (pod)	counted job (pod)	curves
24	`ic-dgb-p24-20261029` (fd47b773)	`ic-dgb-rho-p24-20261029` (d4befe81)	`icv1-fp24-t1973-f876d133`, `icv1-fp24-tm995-ee6c519b`
28	`ic-dgb-p28-20261029` (f0e18393)	`ic-dgb-rho-p28-20261029` (d11f7004)	`icv1-fp28-tm3463-6d2f5699`, `icv1-fp28-t9805-a382b8a8`
32	`ic-dgb-p32-20261029` (1337ff32)	`ic-dgb-rho-p32-20261029` (70d18f78)	`icv1-fp32-tm43175-08b04f34`, `icv1-fp32-tm81759-3ad5e8c5`

Route. `launch-remote` (isolab-eks, EKS Spot c8a.2xlarge, L2 requested), with image `isolab/tasks@sha256:cf0dcc8b…620c` built from `be2969cebf30a0cfb79088fd10094324c3efb3bb` (ecbench `88070375e77648aa12a561ff9ee59e34d469c55e1da56987234811b0956e412c`). Each job launched once; the launch receipts are in `P<b>/{wall,counted}/launch/`.

Cards. `experiment-director` wrote the cards in `cards/`, and `executor-mechanical` ran them unchanged, one size per dispatch:

- `card-launch.md` (sha256 edc0da03…7ccd);
- `card-analysis.md` (sha256 64754a62…cfda).

The pre-launch checks are in `cards/prelaunch-checks.md`. The P2 refusal clause failed there and was ruled on in `cards/RULING-P2.md`. The helpers are in `cards/tools/`, each pinned by sha256 in its card.

Data. The S3 results were checked against each job's `SHA256SUMS`. Large files are in the artifact store; `python3 tools/artifacts/artifacts.py pull research/ic_dg_blocked_20261029` restores them.

Limits

- One host class (c8a.2xlarge, AMD), one image and one binary. Wall time is host-specific: this is a practicality result, not a counted-unit claim.
- Only 28 bits decides. Two curves and 24 targets per curve; the interval comes from 288 paired runs.
- The gain is measured with `targets=stride-renew-deferred-guard` at N_inc and `incremental-gauss`. Other target sources or factor bases are untested.
- `m=2` is the only blocking parameter tested.